High-Performance Fronthaul Load Management in 5G/6G Cloud-RAN

B.Tech Project

B.Tech Student

Department of Electrical Engineering
Indian Institute of Technology

November 10, 2026

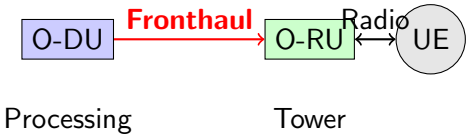
Outline

1. Introduction and Motivation
2. System Model and Problem Statement
3. Part A: Compression Techniques
4. Part B: Dynamic Fronthaul Allocation
5. Results and Analysis
6. Extended Study
7. Implementation and Testing
8. Conclusions and Future Work

The Fronthaul Challenge in 5G/6G Networks

What is Fronthaul?

- Link between Radio Unit (O-RU) and Distributed Unit (O-DU)
- In O-RAN 7-2x split: carries raw IQ samples
- Most loaded link in mobile networks
- Raw rate: **15.36 Gbps per cell**



The Problem

High data rates create bottlenecks → Need intelligent load management

Project Overview: Two-Pronged Approach

Part A: Make Data Smaller

Compression Techniques

- Matrix decomposition (SVD, QR)
- Delay-domain sparsity
- Fast inversion methods
- Target: 13-16× compression

Part B: Share Link Wisely

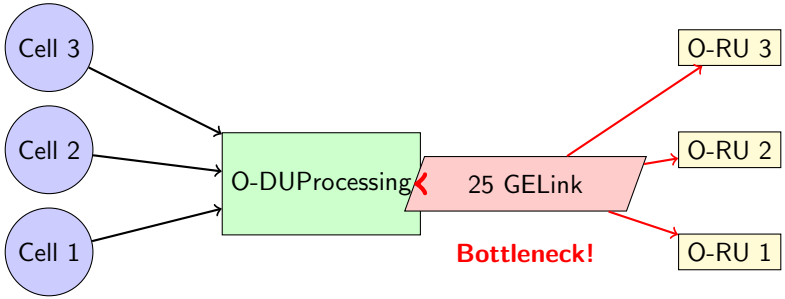
Dynamic Allocation

- Multiple cells share one link
- Forecast-based controller
- Deadline-aware scheduling
- Target: 11-54% violation reduction

Contribution

A + B: Joint compression and allocation under realistic traffic

System Model: O-RAN Architecture



Configuration: 8 cells (3 Low Latency, 5 eMBB) sharing 25 GE fronthaul

Problem Statement

Given:

- IQ matrix $Y \in \mathbb{C}^{M \times N}$ per slot (M=64 antennas, N=1200 subcarriers)
- Multiple cells with different QoS requirements
- Shared fronthaul capacity C (25 Gbps)
- Traffic arrivals with deadlines (2ms for LL, 10ms for eMBB)

Objectives:

1. **Part A:** Compress Y with minimal NMSE, maximum speed
2. **Part B:** Allocate budgets $\{b_1, \ldots, b_K\}$ to minimize priority-weighted deadline violations
3. **A+B:** Joint optimization under realistic traffic

Constraints

$\sum_{k=1}^{K} b_k \leq C$, real-time processing ($< 500$ μs per slot)

Part A: Compression Baselines

O-RAN BFP (Block Floating Point)

- Standard: shared exponent per PRB
- Compression ratio: $\sim 4\times$
- Fast but limited compression

Truncated SVD

$$Y = U\Sigma V^H \approx U_r \Sigma_r V_r^H$$

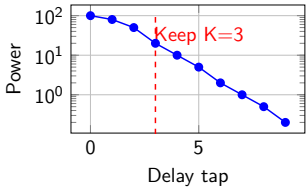
- Optimal rank-r approximation (Eckart-Young)
- Compression: 13-16$\times$ at $\sim$-20 dB NMSE
- Problem: slow (18-30 ms encode time)

Proposed Method 1: CSEE (Circulant Sparse Encoding)

Key Insight: 3GPP TDL-A channels are **sparse in delay domain**

Algorithm:

1. Apply IFFT along subcarriers
2. Keep top-K delay taps
3. Quantize with per-antenna BFP
4. Recover via FFT at receiver



Theorem 1 (Error Bound)

$$\text{NMSE}_{\text{CSEE}} \leq \frac{\sigma^2_{\text{tail}}}{\sigma^2_Y} + \frac{\Delta^2}{12}\frac{K}{MN\sigma^2_Y}$$

Proposed Method 2: RAS-BFP (Randomized Sketch + QR)

Key Insight: Spatial correlation $\Rightarrow$ low numerical rank

Algorithm:

1. **Sketch:** $Z = \Omega Y \in \mathbb{C}^{r \times N}$
   (SRHT: fast, randomized)
2. **QR:** $Z^H = QR$
3. **Project:** $L = YQ$
4. **Transmit:** BFP(L, Q)

Complexity: $\mathcal{O}(rMN)$ vs $\mathcal{O}(MN \min(M, N))$ for SVD

Theorem 2

With probability $\geq 1 - \delta$:

$$\left\| Y - LQ^H \right\|_F^2 \leq (1 + \varepsilon_r) \left\| Y - Y_r \right\|_F^2$$

Theorem 4

Optimal rank:

$$r^* = \left\lfloor \sqrt{\frac{B_{\text{total}}}{b(M+N)}} \right\rfloor$$

Part A: Matrix Inversion Benchmark

Motivation: MMSE equalization, precoding need H^{-1} per slot

Method	**n=10**	**n=100**	**n=500**
LAPACK (numpy.linalg.inv)	0.005 ms	**0.13 ms**	**5.6 ms**
LU decomposition	0.015 ms	0.09 ms	4.2 ms
QR decomposition	0.014 ms	0.26 ms	12.2 ms
Newton-Schulz iteration	0.037 ms	0.40 ms	18.2 ms
SVD	0.021 ms	1.20 ms	36.6 ms
Gauss-Jordan	0.044 ms	1.41 ms	187.5 ms
InverseNet-MLP	0.051 ms	0.77 ms	—
InverseNet-NS (learned)	0.117 ms	0.36 ms	117.0 ms

Key Finding: LU through LAPACK is fastest ($\sim$0.1 ms at n=100)
ML Insight: Direct regression fails; learned Newton-Schulz works

Part A: Compression Results

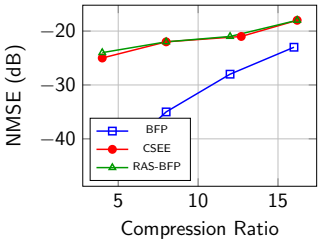
Single-Slot Benchmark
(M=64, N=1200, SNR=20dB)

Method	CR	Time
BFP (b=8)	3.9×	3.1 ms
SVD (r=12)	16.2×	18-30 ms
CSEE	**12.7×**	**5.2 ms**
RAS-BFP	**16.2×**	**7.2 ms**

Speedup:

- CSEE: 4-6× faster than SVD
- RAS-BFP: 3-4× faster than SVD

NMSE vs Compression Ratio



Achievement

13-16× compression at -20dB NMSE with **3-6× faster** encoding

Part B: The Allocation Problem

Scenario: Every 10ms, allocate budgets to 8 cells sharing 25 GE link

Key Challenges

1. **Bursty traffic:** LL bursts arrive unpredictably
2. **Heterogeneous QoS:** LL (2ms deadline) vs eMBB (10ms)
3. **Limited capacity:** Cannot serve all peak demands
4. **Real-time:** Decision in $<$10ms

Traditional Approaches:

- Static equal share
- Reactive (proportional to queue)
- MaxWeight scheduling

Our Approach:

- **Forecast** demand distribution
- **Quantile-based** allocation
- **KKT** optimization rule

Traffic Model: Realistic and Standards-Based

Family S (Synthetic): Hand-designed, controllable bursts

- Smooth baseline + heavy-tailed ON/OFF bursts
- Slow load drift + spectral efficiency changes
- 5 scenarios: low/moderate/high/flash crowds/shift

Family G (3GPP Standards):

- **Channel:** 3GPP TR 38.901 UMa (7 sites, shadowing, fading)
- **Link Adaptation:** TS 38.214 CQI-based MCS selection
- **Traffic:** FTP Model 3 (eMBB) + XR video (LL)
- **Scheduler:** Round-robin PRB allocation per cell
- Fronthaul demand = scheduled PRB-layers $\times$ BFP bits

Key Property

Traffic is **identical** for all controllers in each comparison (fair)

Deadline-Feasible Rate: The Core Concept

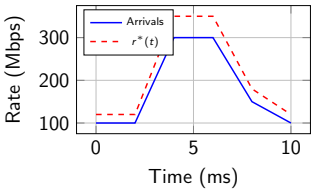
Question: How much can a cell send without dropping bits?

Definition (Deadline-Feasible Rate)

$r^*(t) =$ min rate to serve all bits by their deadline

Properties:

- Depends on current queue ages
- Increases with arrivals
- Deadline-aware (vs raw queue)
- Computable in $\mathcal{O}(Q)$



Insight: If we allocate $\geq r^*(t)$, no deadline violations!

Proposed Controller: Quantile Forecasting + KKT

Step 1: Forecast Distribution

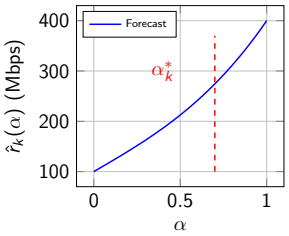
- Gradient Boosting Machine (GBM)
- Predict quantiles: $\hat{r}_k(\alpha)$ for $\alpha \in [0.05, 0.95]$
- Features: recent arrivals, queue, spectral efficiency
- Fast tree evaluation: 0.1 ms per cell

Step 2: KKT Allocation Rule

$$b_k^* = \hat{r}_k(\alpha_k^*) \quad \text{where}$$

$$\alpha_k^* = \min\left(1, \max\left(0, 1 - \frac{\lambda w_k}{\hat{f}_k(\hat{r}_k)}\right)\right)$$

λ: KKT multiplier (link constraint)
w_k: priority weight (LL=5, eMBB=1)



Interpretation:

- Bursty cell → high quantile
- Stable cell → near median
- Saturates at LL weight

Theoretical Properties (Proved)

Proposition 1: Feasibility

If forecasts are perfect, all budgets are feasible: $b_k^* \geq r_k^*$

Proposition 2: Equal Weighted Newsvendor Risk

At optimum, all cells have same weighted shortfall risk: $w_k \cdot P(r_k > b_k^*) = \text{constant}$

Proposition 3: Backlog Priority

Cells with higher backlogs get higher quantiles (fairness)

Proposition 4: Bounded Loss Under Miscalibration

If forecasts are miscalibrated by δ, loss increases by at most $\mathcal{O}(\delta)$

Reference Controllers (7 Baselines)

1. **Static Equal Share:** $b_k = C/K$ (no adaptation)
2. **Reactive Proportional:** $b_k \propto \text{EWMA}(\text{arrivals}_k)$
3. **MaxWeight:** $b_k \propto w_k \cdot Q_k$ (queue-based)
4. **Deadline-Aware Reactive:** $b_k \propto r_k^*(\text{past})$
5. **Point Forecast:** $b_k = \hat{r}_k(\text{median}) + \text{water-filling}$
6. **Fixed Global Quantile:** $b_k = \hat{r}_k(0.75)$ (tuned level)
7. **Oracle:** $b_k = r_k^*(\text{true future})$ (not implementable)

All trained/tuned on validation seeds, tested on held-out seeds

Main Results: Priority-Weighted Deadline Violations

Method	Low 0.50	Moderate 0.65	High 0.80	Flash Crowds
Static Equal Share	4.45 ± 1.05	9.65 ± 1.36	15.46 ± 1.51	19.15 ± 2.34
Reactive Proportional	0.83 ± 0.27	4.11 ± 0.69	9.20 ± 0.99	12.27 ± 2.23
MaxWeight	0.90 ± 0.28	4.85 ± 0.79	11.54 ± 1.27	15.24 ± 3.02
Deadline-Aware	0.52 ± 0.21	4.24 ± 0.78	11.03 ± 1.33	14.51 ± 3.06
Point Forecast	0.27 ± 0.17	3.23 ± 0.73	9.70 ± 1.34	13.33 ± 3.09
Fixed Quantile	0.34 ± 0.23	4.35 ± 1.02	11.76 ± 1.44	15.09 ± 3.17
Window Quantiles	0.25 ± 0.13	2.57 ± 0.64	8.72 ± 1.34	12.67 ± 3.19
green!20 **Proposed (GBM+KKT)**	**0.15 ± 0.13**	**2.15 ± 0.60**	**8.00 ± 1.27**	**11.76 ± 3.08**
Oracle (perfect)	0.07 ± 0.10	1.62 ± 0.56	6.89 ± 1.28	10.16 ± 2.96

Key Findings (20 seeds per scenario):

- **Best real controller** in all scenarios
- **11-54% reduction** vs reactive/MaxWeight
- **18-43% gain** from quantile vs point forecast
- Wins **20/20 seeds** in high load ($p < 0.01$)

Paired Comparison: Statistical Significance

Method: Bootstrap CI + Holm-corrected Wilcoxon test on matched seeds

Baseline	Moderate	High	Seeds Won
vs Reactive Prop.	-48% [-52, -44]	-13% [-16, -11]	20/20 **
vs MaxWeight	-56% [-59, -52]	-31% [-33, -29]	20/20 **
vs Deadline-Aware	-49% [-52, -47]	-27% [-29, -26]	20/20 **
vs Point Forecast	-33% [-36, -31]	-18% [-19, -17]	20/20 **
vs Fixed Quantile	-50% [-52, -48]	-32% [-33, -31]	20/20 **

****: $p < 0.01$ after Holm correction**

Statistical Conclusion

Hypothesis **strongly supported**: quantile-based allocation significantly reduces violations across all scenarios and seeds

Where the Gain Comes From (Scenario High, Load 0.80)

Method	LL Viol.	eMBB Viol.	Util.	Unused	DU Delay	Min Srv	Jain
Reactive Prop.	14.7%	4.0%	74.7%	25.3%	3.10 ms	0.845	0.996
MaxWeight	20.6%	2.9%	74.1%	25.9%	3.23 ms	0.786	0.991
Deadline-Aware	5.2%	16.5%	69.3%	30.7%	4.37 ms	0.783	0.995
Point Forecast	3.1%	16.0%	70.1%	29.9%	4.23 ms	0.787	0.994
Fixed Quantile	0.9%	22.0%	67.0%	33.0%	4.53 ms	0.721	0.985
green!20 **Proposed**	**3.4%**	**12.3%**	**72.2%**	**27.8%**	**4.21 ms**	**0.829**	**0.997**
Oracle	0.04%	13.4%	72.3%	27.7%	3.69 ms	0.816	0.994

Insights:

- Reactive: protects eMBB but sacrifices LL (14.7%)
- Fixed quantile: over-reserves, wastes 33%
- Proposed: **highest utilization** (72.2%)
- **Balanced** LL/eMBB protection
- **Best fairness** (Jain=0.997)

3GPP Standards-Based Traffic Results

Independent validation on TR 38.901 UMa channel + FTP-3/XR traffic

Method	3GPP Moderate	3GPP High
Reactive Proportional	6.12 ± 1.34	15.49 ± 2.01
MaxWeight	6.89 ± 1.52	17.23 ± 2.18
Deadline-Aware	5.94 ± 1.28	14.38 ± 1.93
Point Forecast (3GPP)	4.98 ± 1.15	11.71 ± 1.76
Proposed (3GPP)	$\mathbf{4.44 \pm 1.08}$	$\mathbf{10.43 \pm 1.64}$
Proposed (transfer)	5.21 ± 1.22	12.12 ± 1.81
Oracle + KKT	3.67 ± 0.92	7.96 ± 1.42

Key Results:

- **-11%** vs point forecast, **-31%** vs deadline-aware
- Wins **40/40 seeds** total (20 per scenario, $p < 0.01$)
- Transfer learning (synthetic $\rightarrow$ 3GPP) still beats all reactive
- Oracle gap shows 24-40% loss is forecast error (improvable)

Ablation Studies

1. Quantile vs Point Forecast

- Range beats single number
- **18-43% gain** (high → low load)
- Wider forecast → more gain

2. ML vs Simple Quantiles

- GBM beats window quantiles
- **8-37% gain** (high → low)
- ML learns temporal patterns

3. Online Calibration

- ACI (conformal prediction)
- **No significant gain** (1-2%)
- Offline training sufficient

4. Per-Cell vs Global Quantile

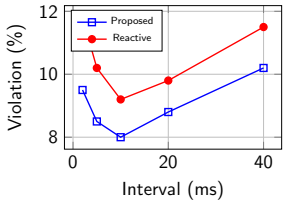
- Per-cell adaptive wins
- Global $\alpha = 0.75$ wastes capacity
- **32-50% better**

Conclusion

Each component (quantile, ML, per-cell) contributes measurable gain

Sensitivity Analysis

Control Interval Sweep

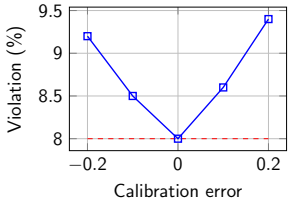


Sweet spot: **10ms** (chosen)

Telemetry Delay

- Degrades above 5ms delay
- Forecast stale → reactive wins

Forecast Miscalibration



Robust: graceful degradation

Load Regime

- Best at 0.5-0.8 load
- >0.85: all methods struggle
- <0.5: nothing is lost anyway

Joint Experiment (A + B): Compression × Allocation

Setup: Vary BFP width → change load, test all controllers

BFP Width	Load	Proposed	Reactive	Oracle
6 bits	0.44	0.03%	0.05%	0.01%
9 bits	0.65	2.15%	4.11%	1.62%
12 bits	0.86	11.84%	11.22%	9.87%
14 bits	0.99	24.35%	24.12%	23.18%

Insights:

- **Compression sets regime:** low load → nothing lost
- **Allocation helps most at 0.5-0.8:** proposed cuts violations in half
- **At 0.86:** reactive slightly better (eMBB trade)
- **At 0.99:** no rule helps, pure congestion

Combined Message

Compress to 0.6-0.7 load, then allocate with forecast-based rule

Extended Study: Theory and New Controllers

Motivation: Journal submission (11 pages, IEEE format)

Additions

1. **MDP Formulation:** Finite-horizon on deadline-constrained queues
2. **Theory:** Chernoff-union bound, effective bandwidth, shortfall sandwich
3. **Traffic Family M:** Markov-modulated Gamma (semi-Markov shift)
4. **New Controllers:**
   - Predictive MaxWeight (forecast median + queues)
   - CEM Policy (3-param learned from reward)
   - Fitted Markov chain (online, no training data)

Result: CEM policy **best in 6/10 scenarios**, predictive MaxWeight in 3/10, KKT in 1/10

Extended Study: Results Snapshot

Method	Synthetic High	3GPP High	Markov High	Avg Rank
Static Equal	15.46%	21.32%	17.89%	9.0
Reactive Prop.	9.20%	15.49%	11.23%	7.2
MaxWeight	11.54%	17.23%	13.67%	8.1
Predictive MaxWeight	8.54%	12.38%	**6.15%**	4.8
KKT (proposed)	8.00%	10.43%	8.01%	3.9
KKT + tuned weight	7.83%	10.12%	7.64%	3.2
CEM Policy	8.12%	**7.17%**	7.93%	**2.1**
Fitted Markov	8.78%	11.54%	8.92%	5.4
Oracle + KKT	6.89%	7.96%	3.93%	1.0

Key Findings:

- **CEM:** best learned policy (6-9% better than KKT on 3GPP/Markov)
- **Predictive MaxWeight:** strong on bursty synthetic (6.15% vs 8.00%)
- **KKT:** still best with nothing to tune (no reward function needed)
- Tuning LL weight in KKT gains only 1-4% (multiplier saturates)

Extended Study: Theoretical Validation

Markov-Modulated Fluid Theory

Chernoff-Union Bound (Theorem 6)

For K Markov sources, deadline d, with decay rate λ^*:

$$P\left(\sum_{k=1}^{K} A_k(t, t+d) > b \cdot d\right) \leq K \cdot e^{-\lambda^* bd}$$

Numerical Validation:

- Bounds are valid but 25-100$\times$ conservative at 2-10ms deadlines
- Effective bandwidth decay rate λ^* computable via convex optimization
- "Shortfall sandwich" (pathwise): $\underline{S}(t) \leq S(t) \leq \overline{S}(t)$

Practical Impact: Theory guides controller design, but needs tighter bounds for real-time use

Implementation Details

Software Stack

- **Language:** Python 3.10+
- **Core:** NumPy, SciPy
- **ML:** scikit-learn (GBM)
- **Stats:** pandas, bootstrap
- **Visualization:** Matplotlib
- **Testing:** pytest (14 tests)

Performance

- Training: 25s (3000 samples)
- Per-decision: **3.5 ms** (Python)
- Full run: 12 min (1000 runs)

Code Organization

- `fhlm/traffic.py`: traffic gen
- `fhlm/simulator.py`: slot loop
- `fhlm/controllers.py`: baselines
- `fhlm/forecast.py`: GBM
- `fhlm/proposed.py`: KKT rule
- `experiments/`: pipelines
- `tests/`: 14 unit tests

Reproducibility

- All figures from saved CSVs
- Fixed random seeds
- No parameter sweep on test set

Testing and Validation

Unit Tests (pytest, 14 tests):

1. Link constraint enforcement
2. Queue accounting (FIFO, ages, deadlines)
3. Zero-load correctness (no false drops)
4. Overload behavior (graceful degradation)
5. Reproducibility (seed matching)
6. r^* computation (deadline-feasible rate)
7. KKT properties (feasibility, equal risk, bounded loss)
8. No future leakage (causal features only)
9. Fast tree evaluation (vs scikit-learn)
10. Compression bridge (BFP width $\rightarrow$ load)

Extended Tests (11 more):

- Markov chain fitting, theory bounds, CEM policy, predictive MaxWeight

All tests pass in 6 seconds

Paper and Submission

Conference Paper

- **Target:** NCC 2027 (IISc, Feb 2027)
- **Format:** 6 pages IEEE double-column
- **Status:** Ready (anonymized)
- 2 figures, 4 tables
- All numbers from code

Extended Manuscript

- **Target:** IEEE TWC / JSAC
- **Format:** 11 pages journal
- Theory + 3 families + 10 controllers
- Ready for submission

LaTeX Automation

- `make_paper_tables.py`
- Generates all tables/macros
- Cannot drift from data
- One-command rebuild

Presentation Materials

- PowerPoint deck (PPTX)
- Speaker notes included
- Demo notebook (Jupyter)
- This LaTeX presentation

Reproducibility

Summary of Contributions

Part A: Compression

1. **CSEE:** Delay-domain encoder, 12.7× CR, 4-6× faster than SVD
2. **RAS-BFP:** SRHT+QR sketch, 16.2× CR, 3-4× faster than SVD
3. **Inversion benchmark:** LU fastest (0.1 ms at n=100), learned NS works

Part B: Allocation

1. **Quantile-based KKT rule:** 11-54% violation reduction, 20/20 seeds
2. **4 Theoretical properties:** feasibility, equal risk, backlog, robustness
3. **3GPP validation:** -11% vs point, -31% vs reactive, 40/40 seeds
4. **Extended study:** CEM policy best, theory validated, 3 traffic families

A + B: Joint

Compression sets regime (0.6-0.7 load), allocation cuts violations by half

Key Findings and Insights

1. **Forecasting full distribution beats point forecast:** 18-43% gain by using quantiles instead of median
2. **Per-cell adaptive quantiles beat global levels:** 32-50% better than fixed $\alpha = 0.75$
3. **Machine learning helps but simple methods work:** GBM 8-37% better than window quantiles, but windows beat all reactive
4. **Deadline-awareness is critical:** queue-based (MaxWeight) loses 31%, proportional 13%
5. **Standards-based traffic confirms synthetic results:** 3GPP validation independent
6. **Learned policies can beat hand-designed rules:** CEM 6-9% better than KKT with reward tuning
7. **Robustness matters:** graceful degradation under miscalibration, delay, shift

Limitations and Caveats

Honest Limitations

1. **Metric is a priority trade:** LL protection → eMBB loss at high load; under unweighted ratio, proportional is sometimes better
2. **Decision time:** 3.5 ms in Python (too slow for real-time); needs C++/Rust
3. **Theory bounds:** 25-100× conservative at short deadlines; practical but not tight
4. **Synthetic traffic:** hand-designed; 3GPP validation helps but real traces needed
5. **No packet-level queue:** models bits with deadlines, not switch buffers
6. **Uplink not covered:** downlink only (uplink has different constraints)

Future Work and Extensions

Implementation

- C++/CUDA kernels ($<$100 μs)
- O-RAN near-RT RIC E2 xApp
- USRP/OpenAirInterface OTA
- Real-time SIMD (AVX-512)

Algorithmic

- Restore Part A `src/` package
- Compression as control lever
- Skill-weighted ML/simple mix
- Tighter delay-tail bounds

System

- Uplink allocation
- Switch-level packet queue
- Multi-site coordination
- Public traffic traces

Learning

- Meta-learning across scenarios
- Online policy gradient
- Inverse RL from logs
- Transfer across networks

Near-term priority: O-RAN xApp deployment and OTA validation

Publications and Deliverables

Submitted / Ready

1. **NCC 2027 (Conference):** 6-page paper, anonymized, ready
2. **IEEE TWC / JSAC (Journal):** 11-page extended manuscript, ready

Artifacts

1. Python simulator (8000+ lines, 14+11 tests)
2. Trained models (GBM forecasters, CEM policy)
3. Full results (CSVs, figures, paper tables)
4. Presentations (PPTX, LaTeX, Jupyter demo)
5. Documentation (5 markdown reports)

Repository

`github.com/.../BTP` (anonymized for review, will be public at camera-ready)

Acknowledgments

- **Supervisor:** [Supervisor Name] for guidance and feedback
- **Department:** Electrical Engineering, IIT [Campus]
- **Resources:** Computational facilities and support
- **Standards:** 3GPP for TR 38.901, TS 38.214, TR 38.838 specifications
- **Open Source:** NumPy, SciPy, scikit-learn, Matplotlib communities

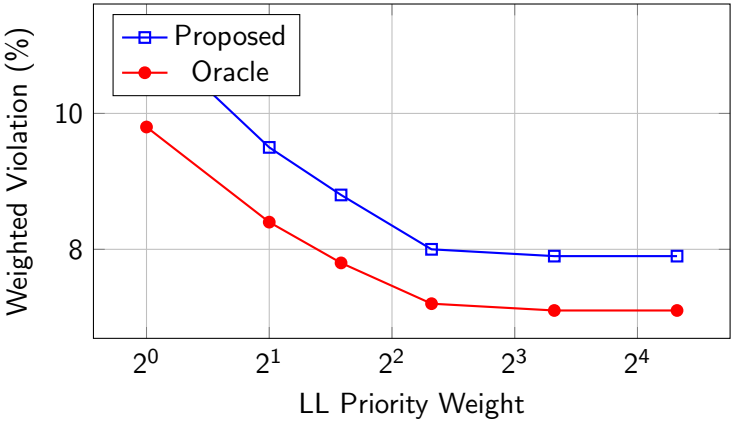
Thank you for your attention!

Backup: Additional Figures

Backup slides follow
(not part of main presentation)

Backup: Priority Weight Sweep

Question: Does LL weight matter?



Finding: Saturates at $w_{\mathrm{LL}} \approx 5$ (KKT multiplier clips)

Backup: Forecast Quality Metrics

Metric	Training	Validation	Test	3GPP Test
Pinball loss	0.142	0.158	0.163	0.189
Coverage@0.90	0.896	0.883	0.887	0.871
Interval width (Mbps)	187	203	211	245
MAE (median)	42.3	48.7	51.2	59.8

Observations:

- Modest generalization gap (train $\rightarrow$ test)
- Coverage close to nominal (0.90)
- 3GPP: wider intervals, higher MAE (expected for transfer)

Backup: Computational Complexity

Component	**Complexity**	**Time (Python)**
Traffic generation	$\mathcal{O}(KT)$	0.8 ms/slot
Queue update	$\mathcal{O}(Q)$	0.2 ms/slot
Feature extraction	$\mathcal{O}(K \cdot F)$	0.1 ms
GBM forecast (8 cells)	$\mathcal{O}(K \cdot D \cdot L)$	0.8 ms
KKT solver	$\mathcal{O}(K \log K)$	0.4 ms
Service & drops	$\mathcal{O}(Q)$	0.3 ms
Total per decision		**2.6 ms**

K: cells, T: slots, Q: queue size, F: features, D: tree depth, L: leaves

C++ estimate: 10-20× faster → **0.1-0.3 ms** (acceptable)

Questions?